

Quản lý bản đồ (Map Management)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (phần bù Schur — đồ thị đồng quan sát có gốc rễ toán học ở đó, và chi phí bất đối xứng giữa tư thế và điểm là gốc của mọi chính sách trong chương này); B/09-paradigm-uoc-luong-back-end (kiến trúc nhiều luồng); B/11-loop-closure-va-nhat-quan-toan-cuc (vấn đề hoà nhập); B/12-relocalization (bản đồ lỗi thời).

TÓM TẮT

Đây là chương bị coi nhẹ nhất trong toàn lĩnh vực và có ảnh hưởng lớn nhất tới việc một hệ SLAM có dùng được trong sản phẩm hay không. Lý do bị coi nhẹ thì dễ hiểu: không ai viết được một bài báo chứng minh chính sách chọn keyframe của mình tốt hơn, vì “tốt hơn” phụ thuộc vào robot chạy bao lâu, ở đâu, và ai trả tiền cho bộ nhớ. Lý do quan trọng thì cũng dễ hiểu: mọi thứ ở Phần B giả định có một bản đồ, và không chương nào ở đó nói bản đồ ấy được tạo ra, cắt tỉa, lưu trữ, cập nhật và chia sẻ thế nào. Nội dung có bốn phần. Một: chính sách chọn và loại — keyframe và landmark — với lập luận rằng cả hai chính sách đều bắt nguồn từ một sự kiện toán học duy nhất ở A/04. Hai: cấu trúc đồ thị của bản đồ, và vì sao đồ thị đồng quan sát không phải một heuristic mà là cấu trúc khác không của một ma trận. Ba: bản đồ sống lâu — nhiều phiên, ghép bản đồ, và bài toán chưa giải là thế giới thay đổi. Bốn: các ràng buộc kỹ thuật thật — nén, tuần tự hoá, phiên bản, và quyền riêng tư. Nếu chỉ nhớ một điều: *tư thế đắt, điểm rẻ* — và gần như mọi chính sách trong chương này là hệ quả của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **keyframe** — một khung hình được giữ lại trong bản đồ như một biến tư thế, khác với các khung chỉ dùng để tracking rồi bỏ. Việc chọn khung nào thành keyframe là quyết định quan trọng nhất trong chương này.
- **đồ thị đồng quan sát (covisibility graph)** — đồ thị có đỉnh là keyframe và cạnh nối hai keyframe cùng nhìn thấy đủ nhiều landmark chung. Nó xác định “vùng lân cận” của một keyframe trong bản đồ.
- **essential graph** — một đồ thị con thừa hơn của đồ thị đồng quan sát, giữ lại cây bao trùm cộng các cạnh mạnh nhất và các cạnh loop closure. Dùng cho tối ưu pose graph, nơi ta cần liên thông chứ không cần mọi cạnh.
- **bản đồ cục bộ (local map)** — tập con của bản đồ mà tracking đang dùng ở thời điểm hiện tại — thường là các keyframe lân cận trong đồ thị đồng quan sát cùng landmark của chúng.
- **làm thưa bản đồ (map sparsification)** — chọn một tập con landmark để giữ sao cho khả năng định vị không giảm đáng kể trong khi kích thước giảm nhiều lần. Bài toán tối ưu tổ hợp, giải bằng heuristic.
- **Atlas** — kiến trúc giữ *nhiều* bản đồ độc lập cùng lúc thay vì một, với cơ chế ghép khi phát hiện chồng lấn. Cách xử lý mất bám tinh tế hơn so với vứt bản đồ và làm lại.
- **map inversion** — tấn công tái tạo lại ảnh gốc từ một bản đồ gồm điểm 3D và descriptor. Đây là một vấn đề quyền riêng tư thật, không phải lo xa.

CÂU HỎI MỞ ĐẦU

- Q1. Giữ nhiều keyframe hơn cho bản đồ chi tiết hơn và tracking bền hơn. Vì sao mọi hệ thống lại tiết kiệm keyframe một cách gắt gao trong khi hào phóng với landmark?
- Q2. Đồ thị đồng quan sát nghe như một cấu trúc dữ liệu tiện lợi do ai đó nghĩ ra. Nêu nguồn gốc toán học của nó, và nói vì sao điều đó có nghĩa là nó không tùy tiện.
- Q3. Robot chạy tám tiếng mỗi ngày trong cùng một toà nhà, ba trăm ngày một năm. Nêu ba thứ sẽ hỏng nếu bạn chỉ đơn giản tiếp tục thêm vào cùng một bản đồ, và nói cái nào khó xử lý nhất.
- Q4. Hệ thống mất bám. Lựa chọn hiển nhiên là vứt bản đồ và khởi tạo lại. Nêu một lựa chọn tốt hơn và nói nó đòi hỏi gì về kiến trúc.
- Q5. Bản đồ của bạn chỉ gồm tọa độ 3D và descriptor nhị phân — không có ảnh nào. Vì sao đó *không* đủ để nói nó không chứa thông tin riêng tư?

1.1 Đặt vấn đề

Mọi chương ở Phần B giả định có một bản đồ. Chúng nói làm sao tạo ra nó (B/10), làm sao dùng nó (B/12), làm sao sửa nó khi đóng vòng (B/11). Không chương nào nói nó *lớn lên thế nào và ta làm gì với việc đó*.

Bài toán gốc phát biểu đơn giản: một hệ SLAM chạy liên tục thêm keyframe và landmark vào bản đồ. Nếu không làm gì, bản đồ tăng tuyến tính theo thời gian, và ba thứ hỏng theo. *Bộ nhớ cạn. Chi phí tính toán tăng* — bundle adjustment cục bộ chậm dần, truy hồi loop closure chậm dần. Và, ít rõ hơn nhưng nghiêm trọng hơn, *chất lượng giảm*: một bản đồ đầy các keyframe gần trùng nhau và các landmark chất lượng kém không tốt hơn một bản đồ gọn gàng — nó tệ hơn, vì tỉ lệ khớp sai tăng theo số ứng viên (B/12 mục 2B).

Ai gặp bài toán này? Chỉ những người chạy hệ thống lâu hơn một chuỗi benchmark. Đây là lý do cấu trúc khiến chương này thiếu tài liệu: EuRoC dài hai phút, KITTI dài vài phút. Không bộ dữ liệu chuẩn nào dài tám tiếng, nên không ai cần giải bài toán này để đăng bài — và mọi người cần giải nó để bán sản phẩm.

Trước khi có các chính sách có nguyên tắc, cách làm phổ biến là ngưỡng thủ công: “tạo keyframe nếu đi quá x mét hoặc xoay quá y độ”. Cách đó hoạt động và nó là cách đúng để bắt đầu, nhưng nó hỏng ở hai chỗ. Nó không thích ứng với cảnh — x mét trong một hành lang hẹp khác hẳn x mét ngoài bãi đỗ xe. Và nó không có cơ chế *loại bỏ*, nên bản đồ vẫn tăng vô hạn, chỉ chậm hơn.

Chương này lập luận rằng phần lớn các chính sách đúng đều suy ra từ một sự kiện toán học duy nhất, đã được thiết lập ở A/04 mục 4C: *chi phí bundle adjustment tăng theo lũy thừa ba của số tư thế nhưng gần tuyến tính theo số điểm*. Tư thế đắt, điểm rẻ. Một khi nắm được điều đó, các chính sách không còn là heuristic tùy tiện mà là hệ quả kinh tế.

TRỰC GIÁC

Hình dung bản đồ như một cuốn album ảnh kèm chú thích, và bạn phải mang nó theo suốt đời.

Mỗi trang album là một keyframe — một chỗ bạn từng đứng, kèm danh sách những gì nhìn thấy từ đó. Mỗi chú thích là một landmark.

Bây giờ một sự kiện quyết định: *thêm một trang thì đắt, thêm một chú thích vào trang có sẵn thì rẻ*. Vì mỗi khi bạn muốn chỉnh lại album cho nhất quán, bạn phải so từng trang với từng trang khác — chi phí tăng theo bình phương số trang. Còn chú thích thì chỉ thuộc về một trang, nên thêm bao nhiêu cũng không làm việc so trang khó hơn.

Từ sự kiện đó suy ra gần hết chính sách. Thêm trang mới chỉ khi nó cho thấy thứ mà các trang cũ không thấy — nếu bạn chỉ nhích sang một bước thì trang mới gần như trùng trang cũ, không đáng. Ngược lại, ghi thật nhiều chú thích vào mỗi trang, vì chúng gần như miễn phí và chúng làm trang đó dễ nhận ra lại hơn.

Rồi có ba chuyện xảy ra theo thời gian, và chúng khó dần.

Album dày lên. Sau vài năm nó nặng quá không mang nổi. Bạn phải xé bớt trang — nhưng xé trang nào? Trang ở chỗ bạn ít khi quay lại, hoặc trang mà các trang lân cận đã ghi hết những gì nó ghi.

Bạn có nhiều album. Mỗi chuyến đi một cuốn. Có lúc bạn nhận ra hai cuốn ghi cùng một nơi, và bạn muốn ghép chúng.

Thế giới thay đổi. Cái nhà trong ảnh đã bị phá. Chú thích của bạn vẫn nói nó ở đó. Bạn không có cách nào biết cho tới khi quay lại — và khi quay lại, bạn không phân biệt được “mình nhớ nhầm chỗ” với “chỗ này đã đổi”.

Chuyện thứ ba là chuyện khó nhất, và nó chưa có lời giải tốt.

1.2 Chính sách chọn và loại

A. Vì sao tư thế đắt còn điểm rẻ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là nền của cả chương.

Từ A/04 mục 4C: sau phần bù Schur, hệ còn lại có kích thước $6m \times 6m$ với m là số tư thế; chi phí giải nó là $O(m^3)$ trong trường hợp xấu (thực tế tốt hơn nhờ độ thưa, nhưng vẫn siêu tuyến tính). Việc khử n landmark tốn $O(n)$ vì khối V là khối chéo.

Hệ quả kinh tế trực tiếp: *gấp đôi số landmark làm chi phí tăng gấp đôi; gấp đôi số keyframe làm chi phí tăng nhiều lần hơn thế.* Với một ngân sách tính toán cố định, tối ưu là giữ ít keyframe và nhiều landmark trên mỗi keyframe.

Có một lý do thứ hai, độc lập và cũng quan trọng: *thông tin*. Hai keyframe cách nhau một bước chân nhìn thấy gần như cùng một thứ từ gần như cùng một chỗ, nên keyframe thứ hai thêm rất ít thông tin — các hàng Jacobian của nó gần tuyến tính phụ thuộc với hàng của keyframe thứ nhất. Nó tốn chi phí đầy đủ mà cho lợi ích gần bằng không. Ngược lại, thêm một landmark mới nhìn từ một hướng mới thật sự thêm ràng buộc.

Hai lý do ấy hội tụ về cùng một chính sách, và đó là lý do nó vững chắc.

B. Tiêu chí tạo keyframe

Ba tiêu chí, và điều đáng chú ý là cả ba đều là *tiêu chí thông tin*, không phải tiêu chí khoảng cách.

Một — đủ parallax so với keyframe gần nhất. Đây là tiêu chí đúng thay cho “đi quá x mét”, vì nó tự thích ứng với độ sâu cảnh: trong hành lang hẹp, một mét cho nhiều parallax; ngoài bãi đỗ xe, một mét cho rất ít. Cách đo: parallax trung vị của các landmark đang theo dõi, đúng đại lượng đã dùng ở B/10 mục 4B.

Hai — tỉ lệ theo dõi giảm. Nếu số landmark cũ còn nhìn thấy được đã giảm dưới một tỉ lệ nào đó, ta đang rời khỏi vùng mà bản đồ hiện tại mô tả, và cần một keyframe để neo vùng mới.

Ba — đã đủ xa về thời gian. Một điều kiện an toàn: dù hai tiêu chí trên chưa kích hoạt, vẫn tạo keyframe sau một số khung nhất định, để tránh trường hợp hệ thống trôi mà không có neo nào.

Tôi không đưa ra các giá trị ngưỡng cụ thể, và đó là có chủ ý. Chúng phụ thuộc vào σ của front-end (B/07), vào độ sâu điển hình của cảnh, vào tốc độ di chuyển, và vào ngân sách tính toán. Cách

đúng là *tính* chúng từ những đại lượng đó — ngưỡng parallax tính theo $B/10$ mục 2C — rồi hiệu chỉnh bằng đo đạc trên dữ liệu của chính mình. Chép một hằng số từ mã nguồn của hệ khác là cách nhanh nhất để có một con số kỳ bí mà ba năm sau không ai biết vì sao.

C. Loại bỏ: phần bị bỏ quên

Tạo thì ai cũng làm; loại thì ít hơn nhiều. Nhưng không có cơ chế loại thì bản đồ vẫn tăng vô hạn.

Loại keyframe dư thừa. Tiêu chí của ORB-SLAM [1] đáng học vì nó đúng nguyên tắc: loại một keyframe nếu phần lớn landmark của nó *cũng* được nhìn thấy bởi ít nhất ba keyframe khác. Lập luận: keyframe ấy không neo landmark nào một cách độc nhất, nên loại nó không mất thông tin về cấu trúc. Chú ý điều này khác hẳn với “loại keyframe cũ” — nó loại theo *mức dư thừa* chứ không theo tuổi, nên một keyframe cũ ở một góc ít ai qua sẽ được giữ, còn mười keyframe mới chồng lên nhau ở một chỗ sẽ bị tía.

Loại landmark chất lượng kém. Một landmark bị loại nếu nó được quan sát quá ít lần so với số lần *lẽ ra* được quan sát (tức nó nằm trong tầm nhìn nhưng không khớp được — dấu hiệu nó là một khớp sai hoặc một vật động), hoặc nếu phần dư của nó liên tục lớn, hoặc nếu bất định của nó không giảm đủ đã quan sát nhiều lần (dấu hiệu suy biến hình học).

Điều đáng nhấn mạnh về chính sách loại landmark: *ngưỡng nên gặt lúc mới tạo và nới dần theo tuổi*. Một landmark vừa được triangulate chưa được kiểm chứng và nên bị loại dễ dàng; một landmark đã sống sót qua hàng trăm quan sát là tài sản quý và nên được bảo vệ. Chính sách này rẻ và hiệu quả bất ngờ, vì phần lớn landmark sai bị loại trong vài khung đầu.

1.3 Cấu trúc đồ thị của bản đồ

A. Đồ thị đồng quan sát và gốc rễ toán học của nó

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những kết nối đẹp nhất trong cây.

Đồ thị đồng quan sát nối hai keyframe khi chúng cùng nhìn thấy đủ nhiều landmark chung. Nó trông như một cấu trúc dữ liệu tiện lợi do ai đó nghĩ ra vì nó hữu ích.

Nó không phải vậy. Quay lại A/04 mục 4C: sau phần bù Schur, ma trận còn lại là

$$S = U - WV^{-1}W^T,$$

và khối (i, i') của S khác không *đúng khi* tồn tại một landmark j mà cả tư thế i lẫn tư thế i' đều quan sát — vì chỉ lúc đó cả W_{ij} lẫn $W_{i'j}$ mới khác không.

Nói cách khác: *đồ thị đồng quan sát chính xác là cấu trúc khác không của S* . Nó không phải một heuristic mà là hình ảnh của một ma trận. (Cửa kiểm chứng a: suy trực tiếp từ công thức Schur.)

Điều này có ba hệ quả thực dụng. *Một:* hai keyframe nối nhau trong đồ thị chính là hai keyframe mà bộ giải *phải* xử lý cùng nhau; tách chúng ra hai bài toán con là làm mất một khối khác không. *Hai:* chọn bản đồ cục bộ cho tracking bằng cách lấy lân cận trong đồ thị đồng quan sát là đúng theo nghĩa toán học — đó chính là tập biến có tương tác trực tiếp. *Ba:* mật độ của đồ thị quyết định chi phí, nên chính sách keyframe ở mục 2 cũng chính là chính sách kiểm soát chi phí.

B. Essential graph và vì sao cần một đồ thị thưa hơn

Đồ thị đồng quan sát đầy đủ có thể rất đặc: trong một căn phòng nhỏ, mọi keyframe nhìn thấy mọi landmark, nên mọi cặp đều nối nhau. Với tối ưu pose graph ở B/11, điều đó làm bài toán đắt không cần thiết.

Essential graph là một đồ thị con giữ lại: cây bao trùm (đảm bảo liên thông), các cạnh có số landmark chung cao nhất, và *mọi* cạnh loop closure. Lập luận: với pose graph, mục tiêu là phân phối sai số trên toàn quỹ đạo, và điều đó cần *liên thông* chứ không cần mọi cạnh. Các cạnh yếu đóng góp ít và có thể bỏ.

Đây là một ví dụ của một khuôn mẫu chung: *dùng biểu diễn đầy đủ cho tối ưu cục bộ, biểu diễn thưa cho tối ưu toàn cục*. Cùng khuôn mẫu xuất hiện ở việc chia luồng ở mục 4A.

C. Làm thưa bản đồ

Với bản đồ dùng cho localization lâu dài, ta không cần *mọi* landmark — ta cần một tập con đủ để định vị ở mọi chỗ với độ chính xác yêu cầu. Bài toán chọn tập con ấy là một bài toán tối ưu tổ hợp, và nó có một cấu trúc đẹp: ta muốn tối đa hoá *độ phủ* (mọi vùng đều có đủ landmark nhìn thấy được) với ràng buộc về kích thước.

Dymczyk và cộng sự [2] đề nghị chọn theo điểm số kết hợp số lần quan sát, độ bền qua các phiên, và độ phủ không gian. Kết quả là một bản đồ nhỏ hơn nhiều lần mà khả năng định vị giảm ít.

Điều đáng nhớ về mặt nguyên tắc: tiêu chí chọn nên là *độ phủ*, không phải *chất lượng trung bình*. Một trăm landmark xuất sắc tụ ở một góc phòng kém hữu ích hơn ba mươi landmark trung bình trải khắp phòng — cùng lập luận với phân bố đặc trưng ở B/07 mục 4A, ở một quy mô khác.

1.4 Bản đồ sống lâu

A. Kiến trúc nhiều luồng

Một hệ SLAM thật có ít nhất ba nhịp độ khác nhau, và cố ép chúng vào một luồng là một lỗi thiết kế.

Tracking chạy ở tần số khung hình — 30 Hz nghĩa là 33 ms cho mỗi khung, và nó *không bao giờ được chặn*, vì camera vẫn gửi khung và robot vẫn di chuyển. *Local mapping* chạy khi có keyframe mới — vài lần mỗi giây, và có thể mất vài trăm mili giây. *Loop closing* chạy hiếm nhưng lâu — có thể vài giây.

Ba nhịp độ ấy chia nhau một bản đồ, và đó là nguồn của phần lớn độ phức tạp kỹ thuật trong một hệ SLAM thật. Ba vấn đề cụ thể: tracking đọc bản đồ trong khi local mapping ghi vào nó; loop closing sắp sửa dịch chuyển các keyframe mà tracking đang dùng; và khi loop closing hoàn thành, các khung đã xử lý trong lúc đó cần được hoà nhập.

CẠM BẮY

Các vấn đề ở mục 4A không xuất hiện trong bài báo nhưng chiếm phần lớn thời gian kỹ thuật của một hệ SLAM thật.

Một bài báo mô tả thuật toán như thể nó chạy tuần tự: tracking, rồi mapping, rồi loop closing. Mã nguồn thật thì đầy hàng đợi, khoá, bản sao, và cờ trạng thái — và phần lớn lỗi khó tái hiện nhất nằm ở đó chứ không nằm trong toán.

Ba hệ quả thực hành. *Một*: đọc mã ORB-SLAM3 hay VINS-Fusion để học *kiến trúc* có giá trị cao hơn đọc bài báo của chúng, và E/23 khuyến nghị điều này. *Hai*: khi thiết kế, hãy quyết định sớm ai sở hữu bản đồ và ai được ghi vào nó — sửa điều này về sau rất đắt. *Ba*: **tính tất định** là một yêu cầu thật chứ không phải một điều xa xỉ; nếu hệ thống cho kết quả khác nhau giữa hai lần chạy trên cùng dữ liệu vì thứ tự luồng khác nhau, bạn không debug được một lỗi từ hiện trường. Đây là chủ đề của D/20 và là một trong những lý do chính đáng nhất để chấp nhận chạy chậm hơn một chút.

B. Nhiều phiên và Atlas

Đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Khi hệ thống mất bám, lựa chọn hiển nhiên là vứt bản đồ và khởi tạo lại (B/10 mục 4C). Nó lãng phí: bản đồ cũ có thể hoàn toàn tốt, chỉ là ta tạm thời không biết mình đang ở đâu trong nó.

Atlas [3] làm khác: khi mất bám, bắt đầu một bản đồ *mới* và giữ bản đồ cũ ở trạng thái không hoạt động. Hệ thống tiếp tục chạy trên bản đồ mới. Song song, nó liên tục thử relocalize vào các bản đồ cũ; nếu thành công, nó *ghép* hai bản đồ bằng phép biến đổi tìm được và tiếp tục với bản đồ hợp nhất.

Ba lợi ích. *Không mất dữ liệu*: công sức lập bản đồ trước đó được giữ. *Phục hồi mềm*: hệ thống vẫn hoạt động ngay cả khi chưa tìm lại được. *Nhiều phiên tự nhiên*: bản đồ từ hôm qua chỉ là một bản đồ không hoạt động trong atlas, và cùng cơ chế ghép hoạt động.

Yêu cầu về kiến trúc: bản đồ phải là một đối tượng *tách rời và ghép được* — có hệ quy chiếu riêng, có thể serialize, và có phép toán ghép. Nhiều hệ thống có bản đồ gắn chặt với trạng thái toàn cục và không làm được điều này. Đây là quyết định thiết kế nên đưa ra sớm.

C. Thế giới thay đổi: bài toán chưa giải

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và là chỗ tôi phải thành thật rằng lĩnh vực chưa có lời giải tốt.

Ba thứ hòng khi chạy lâu trong cùng một môi trường, xếp theo mức khó.

Dễ — *kích thước*. Giải được bằng các chính sách ở mục 2 và mục 3C. Không tầm thường nhưng có lời giải.

Khó vừa — *thay đổi về ngoài theo chu kỳ*. Ánh sáng ban ngày và ban đêm, rèm mở và đóng. Lời giải một phần: lưu *nhiều biểu diễn* cho cùng một địa điểm — mỗi điều kiện một tập descriptor — và chọn theo điều kiện hiện tại. Tồn bộ nhớ và cần một cách phân loại điều kiện, nhưng khả thi. OpenLORIS [4] là một trong ít bộ dữ liệu đo được điều này.

Khó nhất — *thay đổi cấu trúc*. Đồ đạc được sắp lại, một bức tường được dựng, một phòng được cải tạo. Đây là bài toán chưa giải, và lý do nó khó nằm ở một chỗ cụ thể: *không có cách nào phân biệt “bản đồ sai” với “tôi ở sai chỗ” từ bên trong hệ thống*. Cả hai cho cùng triệu chứng — landmark không khớp — và hai kết luận đòi hai hành động ngược nhau: cập nhật bản đồ, hay relocalize.

Các cách tiếp cận hiện có đều là thỏa hiệp. *Quên theo thời gian*: hạ trọng số landmark cũ dần — đơn giản, nhưng nó quên cả những thứ không đổi. *Đếm phiếu*: landmark nào liên tục không khớp qua nhiều lần đi qua thì loại — tốt hơn, nhưng cần nhiều lần đi qua và cần chắc rằng ta thật sự đi qua đúng chỗ, tức cần relocalization đáng tin, tức vòng tròn. *Lưu nhiều phiên bản*: giữ cả bản đồ cũ và mới — tồn bộ nhớ và chuyển bài toán thành việc chọn phiên bản.

Không cách nào giải quyết trường hợp khó nhất, và D/22 xếp đây là một trong tám khoảng trống lớn nhất. Tôi cho rằng nó là khoảng trống *đau nhất* trong triển khai thực tế, vì nó là điểm khác biệt lớn nhất giữa một hệ chạy được một ngày và một hệ chạy được một năm.

1.5 Ràng buộc kỹ thuật thật

A. Nén và tuần tự hoá

Một bản đồ phải lưu được xuống đĩa và tải lại — cho cold start (B/12), cho nhiều phiên, cho cập nhật OTA. Ba yêu cầu thường xung đột.

Kích thước: descriptor chiếm phần lớn. Với ORB, mỗi descriptor là 32 byte; với một bản đồ triệu landmark có vài quan sát mỗi cái, đó là hàng trăm megabyte. Các cách giảm: chỉ lưu một descriptor đại diện cho mỗi landmark thay vì mọi quan sát; lượng tử hoá; hoặc lưu chỉ số từ điển BoW thay vì descriptor đầy đủ — cách cuối làm mất khả năng khớp chính xác nhưng giữ được khả năng truy hồi.

Tốc độ tải: quan trọng cho cold start, nơi người dùng đang chờ. Định dạng nhị phân có cấu trúc phẳng tải nhanh hơn nhiều so với định dạng có nhiều con trỏ phải dựng lại.

Tính tương thích: bản đồ lưu bằng phiên bản phần mềm này phải đọc được bằng phiên bản sau. Đây là yêu cầu thật cho sản phẩm có cập nhật OTA, và nó đòi hỏi *phiên bản hoá định dạng* ngay từ đầu — thêm vào sau rất đau.

B. Quyền riêng tư và map inversion

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là một vấn đề thật chứ không phải lo xa.

Trực giác thông thường: một bản đồ chỉ gồm toạ độ 3D và descriptor thì không chứa ảnh, nên nó không riêng tư. Trực giác đó *sai*.

Pittaluga và cộng sự [5] cho thấy có thể tái tạo lại ảnh nhận ra được từ một bản đồ điểm kèm descriptor. Lý do: descriptor mã hoá về ngoài cục bộ, và với đủ điểm, một mô hình học được có thể ghép chúng lại thành một ảnh. Nói cách khác, *bản đồ SLAM của một căn nhà là một dạng ảnh chụp căn nhà đó*, chỉ ở một mã hoá khác.

Hệ quả cho sản phẩm là thật và đáng nghĩ sớm. Nếu bản đồ được tải lên đám mây — cho xử lý, cho chia sẻ giữa các thiết bị, cho cập nhật — thì đó là truyền dữ liệu riêng tư, với mọi nghĩa vụ pháp lý kèm theo. Nếu bản đồ được lưu trên thiết bị, nó là một mục tiêu.

Các hướng giảm thiểu đang được nghiên cứu: thay điểm bằng *đường* (làm mất thông tin vị trí chính xác trong khi vẫn định vị được), làm nhiều descriptor, hoặc lưu bản đồ ở dạng chỉ dùng được để định vị chứ không dùng được để tái tạo. Không hướng nào hoàn thiện. Điều tối thiểu nên làm: *biết rằng bản đồ là dữ liệu riêng tư và đối xử với nó như vậy*.

CẦN NHỚ

Mọi chính sách trong chương này suy ra từ một câu: tư thế đắt, điểm rẻ.

Chi phí BA là $O(m^3)$ theo tư thế và $O(n)$ theo điểm ($A/04$ mục 4C). Từ đó:

Tạo keyframe **tiết kiệm**, theo tiêu chí thông tin (parallax, tỉ lệ theo dõi) chứ không theo khoảng cách; tạo landmark **hào phóng**.

Loại keyframe theo **mức dư thừa**, không theo tuổi — một keyframe cũ ở góc hiếm người qua đáng giá hơn mười keyframe mới chồng lên nhau.

Chọn bản đồ cục bộ theo **lân cận trong đồ thị đồng quan sát**, vì đó chính là tập biến tương tác trực tiếp trong ma trận S .

Làm thưa bản đồ theo **độ phủ**, không theo chất lượng trung bình.

Và một điều không suy ra từ toán mà từ thực tế: **ngưỡng loại landmark nên gắt lúc mới tạo và nới dần theo tuổi**, vì phần lớn landmark sai lộ ra trong vài khung đầu, còn một landmark sống sót qua hàng trăm quan sát là tài sản đã được kiểm chứng.

❗ Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [3]	Covisibility + essential graph + spanning tree; culling keyframe theo dư thừa; Atlas nhiều bản đồ	Cài đặt đầy đủ nhất của cả chương; đọc mã để học kiến trúc ba luồng
maplab [6]	Nhiều phiên, ghép bản đồ, làm thưa, quản lý dài hạn	Hiếm framework coi quản lý bản đồ là chủ đề chính chứ không phải phụ
[2]	Làm thưa bản đồ theo độ phủ và độ bền qua các phiên	Mục 3C — nguyên tắc chọn theo độ phủ
RTAB-Map	Quản lý bộ nhớ có cấu trúc: working memory và long-term memory	Một cách tiếp cận khác cho bài toán kích thước, lấy cảm hứng từ mô hình trí nhớ người
[5]	Tái tạo ảnh từ point cloud kèm descriptor	Mục 5B — đọc để hết ảo tưởng rằng bản đồ không riêng tư
[4]	Bộ dữ liệu đo thay đổi môi trường theo thời gian	Một trong ít nguồn dữ liệu để nghiên cứu mục 4C

MINI PROJECT SLAM · Chạy một hệ SLAM trong tám tiếng, rồi xem cái gì hỏng · 4 ngày

Mục tiêu	thấy tận mắt ba chế độ hỏng ở mục 4C, vốn không xuất hiện trên bất kỳ benchmark nào vì mọi benchmark đều quá ngắn.
Dữ liệu	tự thu: đặt camera trên một xe đẩy hoặc robot, đi lặp lại một tuyến trong văn phòng hoặc nhà bạn, <i>nhiều lần trong nhiều ngày</i> — ít nhất năm phiên trải trên một tuần, và cố ý dịch chuyển vài đồ vật giữa các phiên.
Các bước	chạy một hệ có sẵn (ORB-SLAM3) trên toàn bộ dữ liệu theo trình tự thời gian; ghi lại theo thời gian: số keyframe, số landmark, bộ nhớ, thời gian mỗi khung của cả ba luồng, tỉ lệ tracking, tỉ lệ relocalization thành công ở mỗi phiên; rồi so bản đồ cuối với bản đồ sau phiên đầu và đếm bao nhiêu landmark của phiên đầu còn sống.
Xong khi	bạn có ba đường cong theo thời gian và một con số. Đường cong: kích thước bản đồ, thời gian mỗi khung, và tỉ lệ tracking. Con số: tỉ lệ landmark từ phiên đầu còn sống sau phiên cuối — đó là <i>tốc độ lão hoá</i> của bản đồ trong môi trường của bạn, và tôi không biết ai đã công bố con số này cho một môi trường thật. Nếu tỉ lệ tracking giảm dần qua các phiên, bạn vừa đo được bài toán ở mục 4C bằng dữ liệu của chính mình.
Nối với	D/19-ben-vung-trong-the-gioi-thuc và D/21-tu-prototype-len-mass-production xây trực tiếp trên kết quả này; D/22-huong-nghien-cuu-con-trong lấy lifelong mapping làm một khoảng trống, và số liệu của bạn là bằng chứng cho nó.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chi phí bất đối xứng: sau phần bù Schur, hệ còn lại có kích thước $6m \times 6m$ và chi phí giải là siêu tuyến tính theo m , trong khi việc khử n landmark chỉ tốn $O(n)$ vì khối V là khối chéo (A/04 mục 4C). Gấp đôi landmark làm chi phí tăng gấp đôi; gấp đôi keyframe làm nó tăng nhiều lần hơn. Có lý do thứ hai độc lập và cũng quan trọng: hai keyframe cách nhau một bước chân cho các hàng Jacobian gần tuyến tính phụ thuộc nên thêm rất ít *thông tin* trong khi tốn chi phí đầy đủ. Hai lý do hội tụ về cùng chính sách, và đó là lý do nó vững.
- A2. Nó *chính xác* là cấu trúc khác không của ma trận $S = U - WV^{-1}W^T$ sau phần bù Schur: khối (i, i') của S khác không đúng khi tồn tại một landmark mà cả hai tư thế cùng quan sát. Nghĩa là nó không tùy tiện — nó là hình ảnh của một ma trận. Ba hệ quả: hai keyframe nối nhau là hai keyframe bộ giải *phải* xử lý cùng nhau; chọn bản đồ cục bộ bằng lân cận trong đồ thị là đúng theo nghĩa toán học vì đó là tập biến tương tác trực tiếp; và mật độ đồ thị quyết định chi phí, nên chính sách keyframe cũng là chính sách kiểm soát chi phí.
- A3. Ba thứ, theo mức khó tăng dần. (a) *Kích thước* — bộ nhớ cạn, tính toán chậm dần, và chất lượng giảm vì tỉ lệ khớp sai tăng theo số ứng viên; giải được bằng chính sách loại và làm thưa. (b) *Thay đổi về ngoài theo chu kỳ* — ánh sáng ngày đêm; giải một phần được bằng cách lưu nhiều

biểu diễn cho cùng địa điểm. (c) *Thay đổi cấu trúc* — đồ đạc sắp lại, tường dựng thêm; **khó nhất và chưa có lời giải**, vì không cách nào phân biệt “bản đồ sai” với “tôi ở sai chỗ” từ bên trong hệ thống — cả hai cho cùng triệu chứng nhưng đòi hai hành động ngược nhau.

- A4. Lựa chọn tốt hơn là *Atlas*: bắt đầu một bản đồ mới và giữ bản đồ cũ ở trạng thái không hoạt động, tiếp tục chạy trên bản đồ mới, đồng thời liên tục thử relocalize vào các bản đồ cũ — nếu thành công thì ghép. Ba lợi ích: không mất công sức đã bỏ ra, hệ vẫn hoạt động ngay cả khi chưa tìm lại được, và nhiều phiên trở thành trường hợp tự nhiên của cùng cơ chế. Yêu cầu kiến trúc: bản đồ phải là một *đối tượng tách rời và ghép được* — có hệ quy chiếu riêng, serialize được, và có phép toán ghép. Nhiều hệ có bản đồ gắn chặt với trạng thái toàn cục và không làm được; đây là quyết định nên đưa ra sớm.
- A5. Vì descriptor mã hoá về ngoài cục bộ, và với đủ điểm, một mô hình học được có thể ghép chúng lại thành một ảnh nhận ra được [5]. Nói cách khác, bản đồ SLAM của một căn nhà là một dạng ảnh chụp căn nhà đó, chỉ ở mã hoá khác. Hệ quả cho sản phẩm: tải bản đồ lên đám mây là truyền dữ liệu riêng tư với mọi nghĩa vụ pháp lý kèm theo; lưu trên thiết bị thì nó là một mục tiêu tấn công. Các hướng giảm thiểu — thay điểm bằng đường, làm nhiều descriptor — đều chưa hoàn thiện. Điều tối thiểu: biết rằng bản đồ là dữ liệu riêng tư và đối xử với nó như vậy.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về bản đồ như một cuốn album ảnh có chú thích mà bạn phải mang theo suốt đời. Mỗi trang là một chỗ bạn từng đứng; mỗi chú thích là một thứ bạn nhìn thấy từ đó. Và có một sự thật quyết định mọi chuyện: *thêm một trang thì đắt, thêm một chú thích thì rẻ*. Vì mỗi lần muốn chỉnh album cho nhất quán, bạn phải so từng trang với từng trang khác — càng nhiều trang càng tốn công theo cấp số. Chú thích thì chỉ thuộc về một trang nên không làm việc so trang khó hơn.

Từ đó suy ra: keo kiệt với trang, hào phóng với chú thích. Chỉ thêm trang mới khi nó cho thấy thứ mà các trang cũ không thấy — nhích sang một bước thì không đáng. Và khi phải xé bớt trang, đừng xé theo tuổi mà xé theo mức *thừa*: một trang cũ chụp cái góc mà không trang nào khác có thì quý, còn mười trang mới chụp cùng một chỗ thì tĩa chín.

Rồi có ba chuyện xảy ra theo năm tháng, và chúng khó dần.

Album dày lên cho tới lúc không mang nổi — chuyện này giải được, chỉ cần kỷ luật xé bớt.

Bạn có nhiều album từ nhiều chuyến đi, và có lúc nhận ra hai cuốn ghi cùng một nơi — chuyện này cũng giải được, chỉ cần biết cách ghép.

Và chuyện thứ ba: *thế giới đổi*. Cái nhà trong ảnh đã bị phá, cái ghế đã dời chỗ. Chú thích của bạn vẫn nói chúng ở đó. Bạn chỉ phát hiện khi quay lại — và khi quay lại, bạn không phân biệt được “mình nhớ nhầm chỗ” với “chỗ này đã đổi”. Hai kết luận ấy đòi hai việc hoàn toàn trái ngược: một bên là đi tìm lại, một bên là sửa album.

Chuyện thứ ba chưa ai giải được, và nó là lý do chính khiến một hệ chạy tốt một ngày lại không chạy tốt được một năm.

Chỗ không rút gọn được: chỗ không rút gọn được là cái vòng luẩn quẩn của chuyện thứ ba: để biết bản đồ sai, bạn phải chắc rằng mình đang ở đúng chỗ; để chắc mình ở đúng chỗ, bạn phải tin bản đồ. Mọi cách tiếp cận hiện có đều là cách phá vòng ấy một cách thô — đếm phiếu qua nhiều lần đi qua, quên dần theo thời gian — và không cách nào thoả đáng.

Một câu: Tư thế thì đắt, điểm thì rẻ — và gần như mọi quyết định về bản đồ là hệ quả của câu đó.

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *đồ thị đồng quan sát là cấu trúc khác không của S* — là một sự kiện đại số suy trực tiếp từ công thức Schur (cửa a), kiểm được bằng cách dựng S và so mẫu khác không với đồ thị. Nó bị bác bỏ nếu có nhân tử nào nổi hai tư thế trực tiếp — chẳng hạn một ràng buộc odometry hay tiền tích phân IMU — vì lúc đó S có thêm khối khác không không đến từ landmark chung. Đây là một điều kiện thật và đáng nêu rõ: *với hệ có odometry, đồ thị đồng quan sát là một tập con của cấu trúc S , không phải toàn bộ*.

Mệnh đề thứ hai — *tư thế đất, điểm rẽ* — suy từ A/04 và kiểm được bằng đo đạc (mini project ở chương 4). Nó mất hiệu lực khi đồ thị đồng quan sát rất đặc, vì lúc đó S đặc và chi phí theo m còn tệ hơn dự đoán — tức mệnh đề đúng theo hướng *mạnh hơn*, không phải yếu hơn.

Mệnh đề thứ ba — *thay đổi cấu trúc chưa có lời giải tốt* — là một phát biểu về trạng thái của lĩnh vực, và nó là mệnh đề tôi ít chắc nhất vì tôi *chưa khảo sát có hệ thống* tài liệu về lifelong SLAM theo chuẩn research-rules.md. Nó bị bác bỏ nếu một hệ duy trì được tỉ lệ relocalization ổn định qua nhiều tháng trong một môi trường thay đổi thật, được kiểm định độc lập. Đây là một trong những mục quan trọng nhất trong 99-chưa-biết.md, và nó nên được nâng lên thành một dự án khảo sát riêng.

Một mệnh đề mềm: chương này khẳng định *quản lý bản đồ bị coi nhẹ trong tài liệu*. Đó là một quan sát về văn hoá nghiên cứu chứ không phải một sự kiện đo được. Nó bị bác bỏ nếu một khảo sát cho thấy tỉ lệ đáng kể các bài SLAM có phần thực nghiệm về hoạt động dài hạn. Tôi cho là không, dựa trên việc các bộ dữ liệu chuẩn đều ngắn, nhưng tôi không đếm (*tôi suy ra, chưa đổi chiều nguồn*).

1.7 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là nguồn của toàn bộ chương: phần bù Schur cho cả đồ thị đồng quan sát lẫn chi phí bất đối xứng. B/09-paradigm-uoc-luong-back-end cho kiến trúc nhiều luồng và pose graph. B/11-loop-closure-va-nhat-quan-toan-cuc nêu vấn đề hoà nhập mà mục 4A ở đây xử lý. B/12-relocalization nêu vấn đề bản đồ lỗi thời mà mục 4C ở đây mở rộng — và vẫn không giải được.

Nối xuống dưới. C/14-hieu-chuan-nhu-mot-nhanh-cua-slam chia sẻ một vấn đề với chương này: cái gì được coi là cố định và cái gì được phép thay đổi theo thời gian. C/16-dense-mapping-va-bieu-dien là chỗ bài toán kích thước trở nên gay gắt hơn nhiều bậc, vì biểu diễn dày tốn bộ nhớ hơn biểu diễn thưa hàng trăm lần. C/18-danh-gia-va-phuong-phap-luan là chỗ ta cần các chỉ tiêu để so sánh hai chính sách quản lý bản đồ — và chương đó phải thừa nhận rằng các chỉ tiêu hiện có không đo được điều này.

Nối xa hơn. D/20-toi-uu-hieu-nang-va-he-thong lấy vấn đề nhiều luồng ở mục 4A làm chủ đề chính, đặc biệt là tính tắt định và ngân sách bộ nhớ. D/21-tu-prototype-len-mass-production là chỗ mục 5 trở thành yêu cầu thật: nén bản đồ cho thiết bị nhúng, phiên bản hoá cho cập nhật OTA, và quyền riêng tư cho nghĩa vụ pháp lý. D/22-huong-nghien-cuu-con-trong lấy lifelong mapping ở mục 4C làm khoảng trống số ba, và tôi cho rằng đó là khoảng trống đau nhất trong triển khai thực tế.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mọi hệ thống tiết kiệm keyframe mà hào phóng landmark? Nêu hai lý do độc lập.
2. Nêu gốc rễ toán học của đồ thị đồng quan sát, và nêu một điều kiện khiến phát biểu đó không còn đúng hoàn toàn.
3. Tiêu chí loại keyframe của ORB-SLAM loại theo mức dư thừa chứ không theo tuổi. Vì sao điều đó quan trọng, và nêu một trường hợp mà loại theo tuổi sẽ sai.
4. Ba nhịp độ của một hệ SLAM là gì, và nêu ba vấn đề kỹ thuật sinh ra từ việc chúng chia nhau một bản đồ.
5. Atlas giải quyết vấn đề gì mà “vứt bản đồ và làm lại” không giải quyết được, và nó đòi hỏi gì về kiến trúc?

6. Vì sao “bản đồ sai” và “tôi ở sai chỗ” không phân biệt được từ bên trong hệ thống, và vì sao điều đó làm lifelong mapping khó?
7. Bản đồ chỉ gồm điểm và descriptor. Vì sao nó vẫn là dữ liệu riêng tư, và hệ quả gì cho thiết kế sản phẩm?

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.